

dμOS Architectural Vision Document 2

Heterogeneous PC Integration, Mega-Node Pooling & Decoupled 4-Daemon Host Architecture

Document Type: System Architecture Spec & Host Component Blueprint

Target Platforms: Linux Workstations, macOS, Windows, Raspberry Pi, ESP32 Mesh

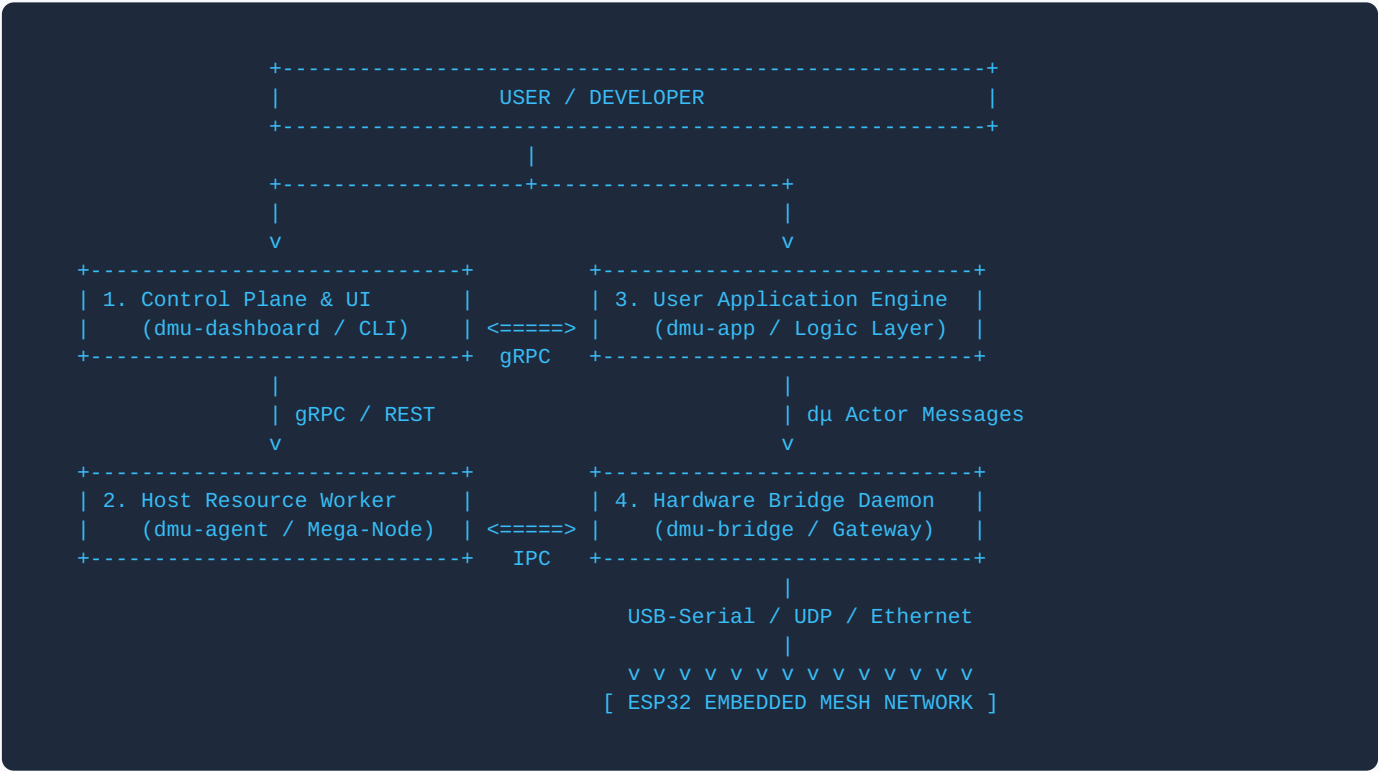
Status: Exploratory / Open Decision Architecture

Author: Tue Wincentz Boas

1. Vision: Extending dμOS to Heterogeneous "Mega-Nodes"

While Document 1 established the embedded micro-fabric running across resource-constrained microcontrollers (e.g., ESP32), **dμOS Document 2** expands the runtime boundary to incorporate desktop PCs, edge servers, and single-board computers (such as Raspberry Pi).

In dμOS, a high-performance PC acts as a **Mega-Node**. Rather than treating microcontrollers and PCs as fundamentally separate computing worlds (which leads to tightly coupled custom serial bridges), dμOS seamlessly abstracts host machines into the global resource pool. To avoid building an unwieldy, brittle monolith on the host side, the PC ecosystem is divided into four cleanly decoupled, single-purpose daemons adhering to Unix design principles.



2. Deep-Dive: The 4 Host Components

Instead of bundling hardware communication, user interfaces, offloading engines, and business logic into a single massive program, dμOS decomposes host responsibilities into four distinct daemons:

Component 1: Hardware Bridge Daemon (`dmu-bridge`) TRANSPORT

Role: Low-level protocol translation and packet routing between the PC OS and the embedded mesh.

- **Functionality:** Reads raw binary dμOS frame packets from physical interfaces (USB-Serial, Ethernet, socket connections to an ESP32 bridge node) and forwards them onto local IPC/UDP sockets inside the PC.
- **Design Constraint:** Written in C++ or Rust for sub-millisecond execution and minimal memory usage. Completely agnostic to user logic, UI rendering, or task scheduling. If the UI or user app crashes, `dmu-bridge` maintains continuous connection to the physical cluster.

Component 2: Host Resource Worker Daemon (`dmu-agent`) COMPUTE

Role: Donating PC CPU cores, RAM, and GPU/NPU acceleration to the dμOS compute pool.

- **Functionality:** Advertises host compute capabilities (e.g., `cores: 16, ram: 32000MB, cuda: true`) into the global dμOS capability directory. Receives offloaded heavy WebAssembly tasks, neural network inferences, or kinematic spatial processing requests dispatched from microcontrollers.
- **Design Constraint:** Can run headlessly as a Linux `systemd` service or background process. Can be paused or throttled dynamically when running on battery power.

Component 3: Control Plane & Management Interface (`dmu-dashboard` / `dmu-ctl`) OBSERVABILITY

Role: Cluster orchestration, topology inspection, and Over-the-Air (OTA) task deployment.

- **Functionality:** Provides a visual Web UI or CLI tool (analogous to `kubectl`). Allows developers to inspect active node health, monitor real-time virtual URIs (e.g., `dmu://nodes/node_03/adc/0`), view live CPU loads, and hot-deploy compiled WASM bytecode to specific nodes.
- **Design Constraint:** Decoupled from hardware connections; can run on a remote laptop, tablet, or browser window miles away from the physical bridge gateway.

Component 4: User Application / Logic Layer (`dmu-app`) BUSINESS LOGIC

Role: High-level orchestration and domain-specific decision rules.

- **Functionality:** Executes user logic (written in Python, Rust, C++, or WASM). Reads virtual hardware paths and dispatches actions to virtual actuators across the fabric.
- **Design Constraint:** Treats dμOS purely as an abstracted API layer. Does not manage network reconnects or low-level frame packing.

3. Architectural Decision Space for PC Integration

Integrating host PCs into a microcontroller mesh introduces technical trade-offs that are preserved as open decision spaces for future implementation.

Decision Area 1: Inter-Daemon PC Communication Protocol OPEN

How do the four host daemons exchange high-throughput messages on the PC locally?

Option A: gRPC over Unix Domain Sockets / Local TCP

- Pros:* Strongly typed Protobuf contracts, cross-language generation (C++, Rust, Go, TypeScript), high performance.
- Cons:* Requires compilation of Protobuf schemas across all host components.

Option B: ZeroMQ / nanomsg Pub/Sub Bus

- Pros:* Extremely lightweight, non-blocking asynchronous event bus architecture.
- Cons:* Less built-in schema validation; requires manual message serialization rules.

Decision Area 2: PC-to-ESP32 Bridge Gateway Topology OPEN

How does the `dmu-bridge` daemon physically interface with the ESP32 network?

Gateway Mode	Physical Connection	Bandwidth & Latency	Best Use Case
USB-Serial Transceiver Node	USB UART to dedicated ESP32 gateway	1–2 Mbps, <2 ms latency	Desktop robotics, local development, direct tethered setups.
UDP Ethernet Gateway	W5500 / Native Ethernet to ESP32	10–100 Mbps, <1 ms latency	Fixed industrial installations, smart building control rooms.
Direct Wi-Fi / Socket Bridge	Wi-Fi UDP / WebSocket connection	10–50 Mbps, 5–15 ms latency	Wireless untethered rovers, drone ground stations, exosuits.

Decision Area 3: Intelligent Task Scheduling Strategy OPEN

How does dμOS decide whether a task executes locally on an ESP32 vs offloading to `dmu-agent` on the PC?

- **Latency-Sensitive Pin Pinning:** Any task with real-time hard deadlines (<10ms control loops, PWM motor drives) is pinned to the physical ESP32 node attached to the peripheral.
- **Compute Cost Heuristics:** Tasks requiring extensive memory allocation (>100KB heap) or heavy floating-point matrix operations automatically route to `dmu-agent` on the PC.

4. Downstream Prompting & Implementation Guide for Coding LLMs

When prompting an LLM to write code for the host-side dμOS ecosystem, feed both Document 1 and Document 2, then use the following structured task template:

System Context: You are acting as a Senior Systems Engineer implementing the host-side PC ecosystem for "dμOS" (Distributed Microcontroller OS).

Input Spec Documents: [Paste Document 1 and Document 2]

Task Focus: [Select one of the 4 Host Components]:

1. "dmu-bridge": Rust or C++ daemon implementing raw USB-Serial/UDP frame parsing into gRPC sockets.
2. "dmu-agent": Lightweight daemon registering host CPU/RAM capabilities into the dμOS CRDT directory and running offloaded Wasm3/WAMR tasks.
3. "dmu-dashboard": React/TypeScript or Terminal TUI dashboard showing live ESP32 cluster topologies and virtual URI metrics.
4. "dmu-app": Python/Rust application SDK for consuming virtual dμOS hardware paths (/dev/sensor/..., /dev/actuator/...).

Constraints:

- Architecture: Strictly decoupled 4-daemon modular design.
- Inter-Process Comm: Protobuf over gRPC / Unix Domain Sockets.

Please provide the project file layout, Protobuf definition file (.proto), and production implementation code for the selected component.